

Übungsblatt 1: Bare-Metal, Polling und Interrupts

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Zielsetzung

In dieser ersten Laborübung machen Sie sich mit der Entwicklungsumgebung und der Hardware vertraut. Sie lernen die praktischen Grenzen von klassischen Polling-Architekturen (Super-Loops) kennen, insbesondere das *Heavy-Load-Problem*. Anschließend lösen Sie dieses Problem, indem Sie die zeitkritischen Aufgaben in deterministische Hardware-Interrupts (Foreground/Background-Architektur) auslagern. Als Hardware dient das Dual-Core-Board **STM32H755**.

Aufgabe 1: Hello World (LED Blink)

Um die Toolchain zu überprüfen, erstellen wir ein einfaches Bare-Metal-Programm auf dem Cortex-M4-Kern.

1. Legen Sie ein neues Projekt für das Board **STM32H755ZI-Q** in der STM32CubeIDE an.
2. Konfigurieren Sie eine LED (z. B. an Pin **PE1**) als **GPIO_Output** und weisen Sie diese dem Cortex-M4 zu (*Pin Context Assignment*).
3. Generieren Sie den Code.
4. Lassen Sie die LED in der `main.c` des CM4-Projekts im Abstand von 500 ms blinken. Nutzen Sie dafür die blockierende Funktion `HAL_Delay()`.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 2: Das Polling-Desaster

Wir provozieren nun einen Timing-Fehler, der zeigt, warum Polling-Loops in der Industrie oft ungeeignet sind.

1. Schreiben Sie eine einfache *Heavy-Load*-Funktion (z. B. eine sinnlose Zählschleife `Heavy_Load(800000)`), die CPU-Last und damit Ausführungszeit simuliert.
2. Rufen Sie diese Funktion in Ihrer `while(1)`-Schleife direkt vor dem LED-Toggle auf.
3. Flashen Sie den Code und beobachten Sie die LED. Erklären Sie, warum das 500-ms-Timing nun zerstört ist.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 3: Rettung durch den Timer

Wir stellen den Determinismus durch Hardware-Präemption wieder her.

1. Aktivieren Sie im Device Configuration Tool (CubeMX) den Timer **TIM2** und wählen Sie als *Clock Source* „Internal Clock“.
2. Setzen Sie im Reiter *NVIC Settings* den Haken bei „TIM2 global interrupt“, damit die Hardware den IRQ zur CPU weiterleitet.
3. Berechnen und setzen Sie die Werte für Prescaler (PSC) und Auto-Reload Register (ARR), sodass der Timer exakt alle 500 ms (2 Hz) überläuft. Nutzen Sie die Formel

$$f_{\text{UPD}} = \frac{f_{\text{CLK}}}{(PSC + 1)(ARR + 1)}$$

Lesen Sie den tatsächlichen Timer-Takt f_{CLK} in der CubeMX-*Clock Configuration* ab (nicht raten!).

4. Generieren Sie den Code. Starten Sie den Timer in der `main()` mit `HAL_TIM_Base_Start_IT(&htim2)`.
5. Entfernen Sie den Toggle-Befehl und das Delay aus der `while(1)`-Schleife (dort verbleibt nur `Heavy_Load()`) und verlagern Sie das Umschalten der LED in die Timer-Callback-Funktion `HAL_TIM_PeriodElapsedCallback`.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 4 (Bonus): Messung am Oszilloskop

In der Industrie wird Timing nicht nur berechnet, sondern physikalisch gemessen.

1. Konfigurieren Sie einen freien Pin (z. B. **PA0**) als **GPIO_Output** und weisen Sie ihn ebenfalls dem Cortex-M4 zu (*Pin Context Assignment*).
2. Messen Sie die Ausführungszeit Ihrer CPU-Last, indem Sie den Pin vor **Heavy_Load()** auf **HIGH** und direkt danach auf **LOW** setzen. Messen Sie die Pulsweite am Oszilloskop.
3. Vergleichen Sie den Jitter (die Schwankung der Ausführungszeitpunkte) der LED-Flanken in der Main-Loop (Aufgabe 2) mit dem Jitter der LED-Flanken im Timer-Interrupt (Aufgabe 3).

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)